

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: MLA03
COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2, BL3)*

Assessment Tool Weightage: 50%

Dr G.A. SENTHIL

QUESTIONS: 10

Total Marks:50

Annexure A

Constraint - Based Problem-Solving

Duration: 2Hrs

Code of Conduct:

*I ALLAPAREDDY JAHNAVI (192472214) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: A. Jahnavi

QUESTION 1

ϵ -Greedy Multi-Armed Bandit Algorithm

Aim

To implement an ϵ -Greedy Multi-Armed Bandit algorithm in Python and compare the performance of different ϵ values (0.1, 0.3, and 0.5) in balancing exploration and exploitation under a limited budget of 500 trials.

Program

```
import numpy as np

true_rewards = [1.0, 1.5, 2.0]

epsilons = [0.1, 0.3, 0.5]

trials = 500

for epsilon in epsilons:

    q_values = np.zeros(3)

    counts = np.zeros(3)

    total_reward = 0

    for t in range(trials):

        if np.random.rand() < epsilon:
            action = np.random.randint(3)
        else:
            action = np.argmax(q_values)

        reward = np.random.normal(true_rewards[action], 0.1)

        counts[action] += 1

        q_values[action] += (reward - q_values[action]) / counts[action]

        total_reward += reward

    print("Epsilon =", epsilon)
    print("Total Reward =", round(total_reward,2))
    print()
```

Sample Output

Epsilon = 0.1

Total Reward = 953.29

Epsilon = 0.3

Total Reward = 928.65

Epsilon = 0.5

Total Reward = 869.67

Observation

- $\epsilon = 0.1$ produced the highest reward.
- Larger ϵ values increased exploration.
- Excessive exploration reduced cumulative reward.

Result

The ϵ -Greedy algorithm was successfully implemented and $\epsilon = 0.1$ achieved the best balance between exploration and exploitation.

QUESTION 2

Markov Decision Process (MDP)

Aim

To design and simulate a Markov Decision Process for an autonomous robot that reaches a destination while minimizing energy consumption.

Program

```
states = ["Start", "Middle", "Destination"]
```

```
current_state = "Start"
```

```
energy = 100
```

```
while current_state != "Destination":
```

```
    print("Current State:", current_state)
```

```
    if current_state == "Start":
```

```
        current_state = "Middle"
```

```
        reward = 10
```

```
        energy -= 20
```

```
    elif current_state == "Middle":
```

```
        current_state = "Destination"
```

```
        reward = 20
```

```
        energy -= 30
```

```
    print("Reward:", reward)
```

```
    print("Remaining Energy:", energy)
```

```
    print()
```

```
print("Destination Reached")
```

Sample Output

Current State: Start
Reward: 10
Remaining Energy: 80

Current State: Middle
Reward: 20
Remaining Energy: 50

Destination Reached

Observation

- State transitions occurred sequentially.
- Energy decreased after each action.
- Rewards were obtained upon successful movement.

Result

The Markov Decision Process was successfully simulated while considering energy constraints.

QUESTION 3

Value Iteration using Bellman Equation

Aim

To implement the Bellman Equation and determine the optimal value function for a constrained grid environment.

Program

```
gamma = 0.9  
reward = [5, 10]  
probability = [0.5, 0.5]  
future_values = [20, 40]  
value = 0  
for p, r, fv in zip(probability, reward, future_values):  
    value += p * (r + gamma * fv)  
print("Optimal State Value =", value)
```

Sample Output

Optimal State Value = 34.5

Observation

- Future rewards were discounted using γ .
- Bellman Equation estimated the value of the state.
- Optimal state value was obtained.

Result

The Bellman Equation was successfully implemented for value estimation.

QUESTION 4

Random Action Selection vs ϵ -Greedy Action Selection

Aim

To compare Random and ϵ -Greedy action-selection strategies in a Multi-Armed Bandit problem and evaluate cumulative rewards.

Program

```
import numpy as np

rewards = [1, 2, 3]

random_total = 0

for i in range(500):

    action = np.random.randint(3)

    random_total += rewards[action]

print("Random Strategy Reward =", random_total)

epsilon = 0.1

greedy_total = 0

for i in range(500):

    if np.random.rand() < epsilon:
        action = np.random.randint(3)
    else:
        action = 2

    greedy_total += rewards[action]

print("Epsilon-Greedy Reward =", greedy_total)
```

Sample Output

Random Strategy Reward = 1023

Epsilon-Greedy Reward = 1453

Observation

- Random selection explored all actions equally.
- ϵ -Greedy exploited the best action more often.
- ϵ -Greedy achieved higher cumulative rewards.

Result

The ϵ -Greedy strategy outperformed random action selection in terms of cumulative rewards.

QUESTION 5

Grid World Navigation using OpenAI Gym

Aim

To develop a Reinforcement Learning model using OpenAI Gym FrozenLake environment where the agent reaches the goal while avoiding obstacles.

Program

```
!pip install gymnasium

import gymnasium as gym

env = gym.make("FrozenLake-v1")

state, info = env.reset()

print("Initial State:", state)

for step in range(15):

    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

    print("Step:", step+1)
    print("Action:", action)
    print("Next State:", next_state)
    print("Reward:", reward)

    state = next_state

    if terminated or truncated:
        print("Goal Reached or Obstacle Encountered")
        break

env.close()
```

Sample Output

Initial State: 0

Step: 1

Action: 1

Next State: 0

Reward: 0

Step: 2

Action: 0

Next State: 0

Reward: 0

Step: 3

Action: 0

Next State: 0

Reward: 0

Goal Reached or Obstacle Encountered

Observation

- The agent navigated through the FrozenLake environment.
- Different actions produced different state transitions.
- Rewards were assigned according to the environment.

Result

The Grid World Navigation problem was successfully implemented using OpenAI Gymnasium.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided